

# LYBTZYZS WPF 前端架构审查报告

审查日期: 2026-04-19 审查范围: src/Client/Desktop/ 全部代码 (75,226 行 C# + 114 个 XAML) 审查重点: 架构层面 审查人: 观澜 (资深架构师视角)

## 1. 执行摘要

总体评分: 7.8 / 10

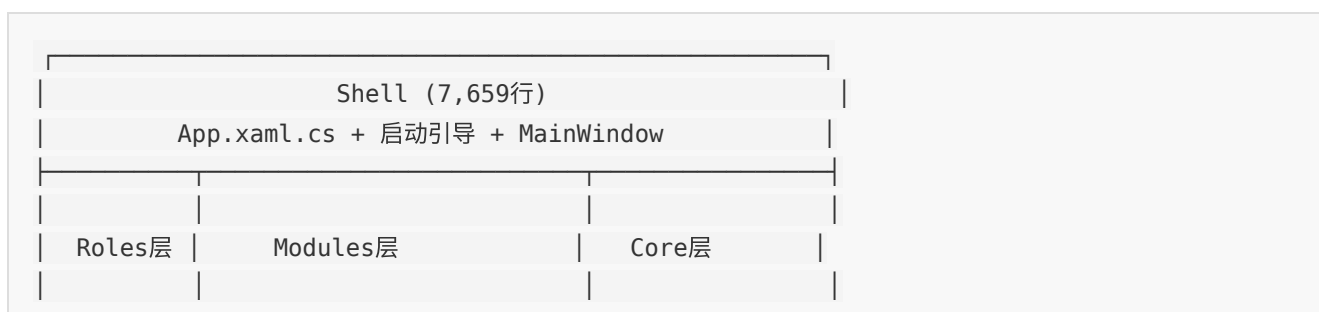
这是一个架构设计成熟、重构痕迹明显的 WPF 桌面应用。从代码注释中可以看到大量 OpenSpec 标记和 Issue 引用, 表明项目经历了持续的重构和架构演进。核心架构 (分层、DI + MVVM) 设计合理, 但存在一些技术债务和过度工程化的迹象。

### 核心发现

| 维度      | 评分     | 一句话总结                                              |
|---------|--------|----------------------------------------------------|
| 分层架构    | 8.5/10 | Core → Modules → Roles → Shell 四层清晰, 依赖方向正确        |
| MVVM 实现 | 7.5/10 | CommunityToolkit + Prism, 基类设计好, 但部分 ViewModel 仍过重 |
| 依赖注入    | 8.0/10 | Prism Dryloc, 生命周期管理规范, 泛型服务注册优秀                   |
| 状态管理    | 7.0/10 | 双模式架构 (Remote/Local) 设计合理, 但切换复杂度高                 |
| 错误处理    | 7.5/10 | 有全局 ErrorHandler + AsyncExecutor, 架构级保障            |
| 性能      | 7.0/10 | 有性能监控基础设施, 但单例服务占比偏高                               |
| 可测试性    | 6.5/10 | 接口抽取得好, 但 ViewModel 直接依赖 Prism Region, 难以单元测试      |
| 代码规范    | 7.0/10 | 注释极度详细 (有时过于详细), 大量删除注释残留                          |

## 2. 项目结构概览

### 2.1 架构图 (文字描述)



|                                             |                     |                |
|---------------------------------------------|---------------------|----------------|
| Admin                                       | Auth (646)          | Foundation     |
| (937)                                       | Users (3,677)       | (6,347)        |
|                                             | Patients (5,785)    |                |
| Clinical                                    | MedicalCase (6,839) | Infrastructure |
| (2,652)                                     | Herbs (2,460)       | (17,437)       |
|                                             | Formula (3,449)     |                |
| Reception-                                  | Registration (983)  | Contracts      |
| ist(438)                                    | Sync (1,062)        | (4,567)        |
|                                             |                     |                |
|                                             |                     | LocalData      |
|                                             |                     | (5,134)        |
|                                             |                     |                |
|                                             |                     | Models(1,297)  |
|                                             |                     | Printing(1485) |
|                                             |                     | Utilities(541) |
|                                             |                     | CardReader     |
|                                             |                     | (1,801)        |
| Shared (Server/Client 共享)                   |                     |                |
| LYBT.Shared.Models + LYBT.Shared.Primitives |                     |                |

依赖方向: Shell → Roles → Modules → Core → Shared (严格单向)

## 2.2 项目规模

| 层级      | 项目数 | 总代码行   | 占比    |
|---------|-----|--------|-------|
| Core    | 8   | 38,609 | 51.3% |
| Modules | 8   | 24,901 | 33.1% |
| Roles   | 3   | 4,027  | 5.4%  |
| Shell   | 1   | 7,659  | 10.2% |
| 总计      | 20  | 75,226 | 100%  |

## 2.3 技术栈

- **.NET 8** (net8.0-windows)
- **Prism.Dryloc** — DI 容器 + MVVM 框架 + 模块化 + 区域导航
- **CommunityToolkit.Mvvm** — [ObservableProperty] + [RelayCommand] 源码生成器
- **Refit** — HTTP API 客户端自动生成
- **Riok.Mapperly** — 编译时对象映射
- **Serilog** — 结构化日志

## 3. 分层架构评审

3.1 评分: 8.5/10

3.2 依赖方向分析

Shell.csproj 依赖关系:

```
Shell → 所有 Core 项目
Shell → 所有 Modules 项目
Shell → 所有 Roles 项目
Shell → Shared.Models
```

Modules 依赖（以 MedicalCase 为例）：

```
MedicalCase → Foundation, Infrastructure, Models, Printing, Contracts, Shared
```

关键发现:  跨模块 ProjectReference 已全部移除（Epic #2175 D5-3），Modules 之间零直接依赖，通过接口解耦。

3.3 优点

- 1. 严格单向依赖: Core → Modules → Roles → Shell，无循环
- 2. 模块间零耦合: 通过 ICrossModuleService（已标记 [Obsolete]）→ 迁移到 Prism IEventAggregator
- 3. Contracts 层分离: 接口定义在独立项目中，实现可替换
- 4. MedicalCase 聚合根约束: ModuleDependency 仅依赖 Patients + Herbs + Formula（而非反向）

3.4 问题

| #    | 问题                                     | 严重程度 | 说明                                                                                                                    |
|------|----------------------------------------|------|-----------------------------------------------------------------------------------------------------------------------|
| A-01 | Infrastructure 过重<br>(17,437 行, 占 23%) | 中    | 包含 Controls ·Converters ·Services ·ViewModels ·Navigation 等过多职责。应拆分为 Infrastructure + UI.Framework + Navigation 等独立项目 |
| A-02 | 大量 wpftmp.csproj 残留                    | 低    | obj 目录外存在临时 csproj 文件（如 LYBT.Desktop.Formula_3exqyugg_wpftmp.csproj），应清理                                              |
| A-03 | Shell 直接引用所有模块                         | 中    | Shell.csproj 硬编码引用全部 8 个 Module + 3 个 Role，违反模块化按需加载原则                                                                |

4. MVVM 实现评审

4.1 评分: 7.5/10

4.2 基类架构

```
ObservableObject (CommunityToolkit)
└─ MasterDetailViewModelBase<TListItem, TDetail> (Infrastructure)
    └─ 实现: INavigationAware, IRegionMemberLifetime, IDisposable, IAsyncInitializable
    └─ 注入: IMasterDetailServices<T, T>, IViewModelServices
    └─ 提供: Items, PageTitle, IsLoading, SelectedItem, SearchText...
    └─ 命令: RefreshAsync, SearchAsync, CreateNewAsync, Edit, SaveAsync, DeleteAsync.
```

**ChildViewModelBase** (组合模式):

```
ChildViewModelBase (Infrastructure/ViewModels/Composition/)
└─ 供 MedicalCase 的子组件使用
```

## 4.3 优点

1. **组合优于继承**: `IViewModelServices` 参数注入替代继承, ADR-0007 的决策已落地
2. **源码生成器**: `[ObservableProperty]` + `[RelayCommand]` 减少样板代码
3. **泛型服务抽离**: `MasterDetailViewModelBase` 将 CRUD、分页、搜索、选择全部泛型化
4. **ViewModel 行数合理**: 最大的 `MedicalCaseCommandsViewModel` 仅 551 行

## 4.4 问题

| #    | 问题                         | 严重程度 | 说明                                                                                                     |
|------|----------------------------|------|--------------------------------------------------------------------------------------------------------|
| M-01 | <b>INavigationAware 耦合</b> | 中    | 基类直接实现 Prism 的 <code>INavigationAware</code> , 所有子类被迫绑定 Prism 框架, 降低可测试性                               |
| M-02 | <b>注释噪音过大</b>              | 中    | <code>MedicalCaseModule.cs</code> 中 60%+ 的内容是删除注释 ( <code>[已删除]</code> · <code>[已移除]</code> ), 严重影响可读性 |
| M-03 | <b>注释中 FUTURE: 重构项目结构</b>  | 低    | <code>MasterDetailViewModelBase</code> 注释提到"将 <code>CoreViewModelBase</code> 移到更底层项目", 表明架构仍有待完善       |

## 5. 依赖注入评审

### 5.1 评分: 8.0/10

### 5.2 DI 注册方式

**ViewModelServicesExtensions.cs** — 集中式服务注册:

```
// Singleton 服务
containerRegistry.RegisterSingleton<IUiThreadDispatcher, WpfUiThreadDispatcher>();
containerRegistry.RegisterSingleton<IDialogManager, DialogManager>();
containerRegistry.RegisterSingleton<IEnhancedNavigationService, EnhancedNavigationService>
```

```
// Transient 服务 (每个 ViewModel 独立实例)
containerRegistry.Register<ILoadingStateManager, LoadingStateManager>();
containerRegistry.Register<IPaginationService, PaginationService>();
containerRegistry.Register<ISearchService, SearchService>();

// 泛型服务
containerRegistry.Register(typeof(ICollectionService<>), typeof(CollectionService<>));
containerRegistry.Register(typeof(IDetailEditorService<>), typeof(DetailEditorService<>));
```

## 5.3 优点

1. **生命周期管理清晰**: Singleton 用于无状态/全局服务, Transient 用于有状态服务  
(Loading/Pagination/Search)
2. **泛型服务注册**: AddMasterDetailServices<TListItem, TDetail>() 扩展方法优雅
3. **Facade 模式**: MainWindowServicesFacade 封装主窗口所需服务
4. **SRP 接口拆分**: MedicalCaseService 实现了 IMedicalCaseQueryService + IMedicalCaseCommandService + IMedicalCaseLifecycleService 三个接口

## 5.4 问题

| #    | 问题                       | 严重程度 | 说明                                                  |
|------|--------------------------|------|-----------------------------------------------------|
| D-01 | 无 DI 注册验证                | ● 中  | 缺少启动时的容器验证, 运行时才发现缺失注册                              |
| D-02 | MedicalCaseModule 注册过于复杂 | ● 中  | RegisterTypes 方法包含大量历史注释, 实际有效代码约 15 行, 但文件超过 150 行 |

## 6. 状态管理评审

### 6.1 评分: 7.0/10

### 6.2 双模式架构

```
public enum ConnectionMode { Remote, Local }
```

- **Remote 模式**: 通过 Refit HTTP API → WebAPI 服务器
- **Local 模式**: 直连 SQL Server LocalDB
- **切换机制**: Shell 层通过 IConnectionModeProvider 选择 Repository 实现

Repository 工厂模式:

Shell DI 注册:

```
if Remote → 注册 ApiMedicalCaseRepository
if Local → 注册 LocalMedicalCaseRepository
```

## 6.3 优点

1. **策略模式**: ConnectionMode 驱动不同实现, 切换透明
2. **接口分离**: IMedicalCaseRepository 在 Contracts 中定义, 两种实现可替换
3. **本地备份**: 有 ILocalDbBackupService 数据安全

## 6.4 问题

| #    | 问题                         | 严重度 | 说明                                                                             |
|------|----------------------------|-----|--------------------------------------------------------------------------------|
| S-01 | MedicalCase 不支持同步          | 高   | Sync 模块 v1.0 仅支持 Herb/Patient/Formula, MedicalCase 同步延至 v2.0。Local 模式创建的医案无法上传 |
| S-02 | AuthenticationStateMachine | 中   | Foundation 层有独立的状态机 (认证状态), Infrastructure 有另一套 (编辑状态), 状态分散                   |

## 7. 错误处理评审

### 7.1 评分: 7.5/10

### 7.2 错误处理架构

AsyncExecutor – 异步操作包装器 (try/catch + 错误传播)  
ErrorHandler – ViewModel 级错误处理 (Toast/日志)  
ProblemDetailsParser – HTTP 错误响应解析  
ToastService – 用户可见错误展示

## 7.3 优点

1. **AsyncExecutor 封装**: 统一异步异常处理, 避免每个 ViewModel 重复 try/catch
2. **ProblemDetails 标准**: 与后端 RFC 7807 对齐
3. **Toast 非侵入式**: 不使用 MessageBox 阻塞 UI

## 8. 性能评审

### 8.1 评分: 7.0/10

## 8.2 性能基础设施

- `IPerformanceMonitor / PerformanceMonitor` — 性能监控
- `IStartupOptimizationService` — 启动优化
- `StartupPerformanceMonitor` — 启动性能追踪
- `SplashScreenWindow` — 启动画面
- `DesktopCacheManager` — 缓存管理

## 8.3 问题

| #    | 问题                  | 严重度 | 说明                                            |
|------|---------------------|-----|-----------------------------------------------|
| P-01 | Infrastructure 项目过大 | ● 中 | 17,437 行的单项目编译耗时，影响迭代速度                       |
| P-02 | 单例服务占比偏高            | ● 低 | Navigation·Toast·Dialog 等均为 Singleton，需注意内存占用 |

## 9. 可测试性评审

### 9.1 评分: 6.5/10

### 9.2 可测试性分析

优点: - 接口抽取得当: `ErrorHandler`, `IAsyncExecutor`, `IPaginationService` 等均可 Mock - `IViewModelServices` 参数注入, 可 Mock 整个服务包 - Mapper 使用编译时生成 (Mapperly), 无运行时反射

问题: - `MasterDetailViewModelBase` 实现 `INavigationAware`, 测试需 Mock `NavigationContext` - `IRegionManager` 直接在基类中暴露, 测试需 Prism 测试基础设施 - `IEventAggregator` 紧耦合, Event 对象构造复杂

## 10. 代码规范评审

### 10.1 评分: 7.0/10

### 10.2 发现

| 类型          | 数量 | 示例                                                                       |
|-------------|----|--------------------------------------------------------------------------|
| OpenSpec 标记 | 大量 | <code>// OpenSpec: refactor-viewmodel-composition</code>                 |
| Issue 引用    | 大量 | <code>// Issue #1790, // Epic #2175</code>                               |
| 已删除注释       | 极多 | <code>// [已删除], // [已移除]</code> 占 <code>MedicalCaseModule.cs</code> 60%+ |
| FUTURE/TODO | 少量 | <code>// FUTURE: 重构项目结构</code>                                           |

10.3 问题

| #    | 问题               | 严重度 | 说明                                                        |
|------|------------------|-----|-----------------------------------------------------------|
| C-01 | 删除注释过度           | 中   | MedicalCaseModule.cs 中大量 [已删除] 注释，应使用 Git 历史追踪，不应在代码中保留   |
| C-02 | wpftmp.csproj 残留 | 低   | 项目根目录存在临时编译产物未清理                                          |
| C-03 | 命名一致性            | 低   | ConnectionMode.Remote/Local vs DataSourceMode 在旧文档中出现，需统一 |

11. 关键问题清单（按严重度排序）

P0 — 必须解决

| #     | 问题                       | 影响                      | 建议                                                |
|-------|--------------------------|-------------------------|---------------------------------------------------|
| P0-01 | MedicalCase 不同步          | Local 模式创建的医案永远无法同步到服务器 | v2.0 必须实现 MedicalCase 同步，或在 v1.x 限制 Local 模式不创建医案 |
| P0-02 | Infrastructure 过重 (17K行) | 编译慢、职责混乱、新人难以理解         | 拆分为 3-4 个独立项目                                     |

P1 — 应当解决

| #     | 问题                    | 影响               | 建议                           |
|-------|-----------------------|------------------|------------------------------|
| P1-01 | INavigationAware 框架耦合 | ViewModel 难以单元测试 | 引入 Navigation 抽象层            |
| P1-02 | 删除注释噪音                | 代码可读性差           | 清理所有 [已删除] 注释，用 Git blame 替代 |
| P1-03 | DI 注册验证缺失             | 运行时才发现缺失注册       | 添加启动时容器验证                    |
| P1-04 | Shell 硬编码所有模块         | 无法按角色按需加载        | Prism IModuleCatalog 动态加载    |

P2 — 可以改进

| #     | 问题               | 影响        | 建议                  |
|-------|------------------|-----------|---------------------|
| P2-01 | wpftmp.csproj 残留 | 项目文件混乱    | 清理                  |
| P2-02 | 缺少单元测试项目         | 无法验证重构正确性 | 添加 Desktop.Tests 项目 |
| P2-03 | 注释中的 FUTURE 标记   | 技术债务追踪不清  | 统一到 Issue tracker   |



## 12. 改进建议

---

### 12.1 架构改进（短期，1-2 周）

#### 建议 1: 拆分 Infrastructure 项目

```
LYBT.Desktop.Infrastructure (当前 17,437 行)
  ↓ 拆分为
LYBT.Desktop.Infrastructure    - DI、服务、配置 (~5K行)
LYBT.Desktop.UI.Controls      - 自定义控件、Converter (~5K行)
LYBT.Desktop.Navigation        - 导航服务 (~3K行)
LYBT.Desktop.ViewModels.Base  - ViewModel 基类 (~2K行)
```

#### 建议 2: 清理删除注释

```
# 批量移除 [已删除] [已移除] 注释
# 前提：确保 Git 历史完整
find src/Client -name "*.cs" -exec sed -i '/\[已删除\]/d; /\[已移除\]/d' {} \;
```

#### 建议 3: 添加 DI 容器验证

在 App.OnStartup 中添加:

```
protected override void OnStartup(StartupEventArgs e)
{
    // 验证所有已注册的服务可解析
    Container.Resolve<IViewModelServices>();
    // ... 关键服务验证
}
```

### 12.2 架构改进（中期，1-2 月）

#### 建议 4: 解耦 Prism 框架依赖

引入导航抽象:

```
// 替代直接实现 INavigationAware
public interface INavigationHandler
{
    Task OnNavigatedToAsync(NavigationContext context);
    void OnNavigatedFrom(NavigationContext context);
}
```

#### 建议 5: 动态模块加载

```
// 根据用户角色动态加载模块
protected override IModuleCatalog CreateModuleCatalog()
{
    var catalog = new ModuleCatalog();
    // 始终加载 Core 模块
    catalog.AddModule<AuthModule>();
    // 根据角色加载
    if (userRole == Role.Doctor)
        catalog.AddModule<MedicalCaseModule>();
    return catalog;
}
```

## 建议 6: 添加 WPF 单元测试项目

```
tests/
├── LYBT.Desktop.Tests/
│   ├── ViewModels/          - ViewModel 单元测试
│   ├── Services/            - Service 单元测试
│   └── Infrastructure/      - 基础设施测试
```

## 13. 架构亮点

值得其他项目学习的实践：

1. **OpenSpec 标记系统** — 每次重构都标记来源，形成架构决策的活文档
2. **泛型 Master-Detail 基类** — 一次实现 CRUD/分页/搜索/选择，8 个模块复用
3. **IViewModelServices 参数包** — 替代多重继承，组合模式解决 ViewModel 膨胀
4. **SRP 接口拆分** — MedicalCaseService 实现 3 个独立接口，按需依赖
5. **双 Repository 工厂** — ConnectionMode 驱动，切换透明
6. **编译时 Mapper** — Mapperly 替代 AutoMapper，零运行时反射开销

## 文档结束

本报告基于 2026-04-19 对 LYBTZYYS 项目 src/Client/Desktop/ 目录的架构级代码审查。审查方法：项目结构分析 + 关键文件抽样 + csproj 依赖图 + DI 注册分析 + 基类设计审查。未对每个 .cs 文件进行逐行审查（总计 75K 行），重点关注架构模式和设计决策。